

SLATE User Manual

Strategy Learning & Autonomous Trading Engine
Version 2.0
April 2026

SLATE User Manual

Strategy Learning & Autonomous Trading Engine

****Version 2.0****

****April 2026****

Table of Contents

- [Introduction to SLATE](#1-introduction-to-slate)
- [System Architecture](#2-system-architecture)
- [Getting Started](#3-getting-started)
- [The Dashboard Interface](#4-the-dashboard-interface)
- [Discovery Systems](#5-discovery-systems)
- [Strategy Types Explained](#6-strategy-types-explained)
- [Testing Methodologies](#7-testing-methodologies)
- [Practical Usage Examples](#8-practical-usage-examples)
- [Understanding Results](#9-understanding-results)
- 10. [Advanced Configuration](#10-advanced-configuration)
- 11. [Troubleshooting](#11-troubleshooting)
- 12. [Best Practices](#12-best-practices)

1. Introduction to SLATE

What is SLATE?

SLATE (Strategy Learning & Autonomous Trading Engine) is an advanced AI-powered system designed to discover, test, and evolve trading strategies automatically. Unlike traditional trading systems that require manual strategy development, SLATE uses machine learning and genetic algorithms to explore thousands of strategy variations and identify those with the highest potential for profitability.

Core Philosophy

SLATE operates on several key principles:

****Continuous Discovery****: SLATE never stops learning. It runs 24/7, generating and testing new strategies based on what it has learned from previous tests.

****Realistic Testing****: All strategies are tested against real historical market data, ensuring that backtesting results reflect actual trading conditions rather than idealized scenarios.

****Self-Evolution****: The system improves itself over time by learning which strategy types perform best and evolving new candidates based on successful patterns.

****Robustness First****: SLATE prioritizes strategies that perform consistently across different market conditions rather than those that show impressive results in only one scenario.

What SLATE Can Do For You

- ****Automatically Discover Strategies****: No need to manually code trading ideas. SLATE generates and tests thousands of strategies automatically.
- ****Test Across Market Conditions****: Evaluate how strategies perform in trending, ranging, volatile, and quiet markets.
- ****Identify High-Potential Candidates****: Find strategies with strong risk-adjusted returns using metrics like Sharpe ratio, maximum drawdown, and consistency.
- ****Evolve and Improve****: Take promising strategies and automatically create variations to potentially improve performance.
- ****Reduce Research Time****: What would take months of manual testing can be accomplished in days or even hours.

Who Should Use SLATE?

- ****Quantitative Traders****: Looking for systematic ways to generate and test strategy ideas
- ****Algorithmic Traders****: Seeking to automate strategy discovery and testing
- ****Trading Firms****: Needing to rapidly explore large strategy spaces
- ****Individual Traders****: Wanting to leverage AI for strategy development without programming expertise
- ****Researchers****: Exploring market patterns and testing hypotheses about market behavior

2. System Architecture

High-Level Overview

SLATE consists of several interconnected components that work together to provide continuous strategy discovery and testing:

Before installing SLATE, ensure your system meets these requirements:

****Operating System**:**

- macOS 10.15+
- Linux (Ubuntu 18.04+, Debian 10+, or similar)
- Windows 10+ with WSL2

****Hardware**:**

- Minimum: 4GB RAM, 2 CPU cores
- Recommended: 8GB+ RAM, 4+ CPU cores
- Storage: 10GB free space (for historical data and test results)

****Software**:**

- Python 3.8 or higher
- Modern web browser (Chrome, Firefox, Safari, or Edge)
- Internet connection (for fetching market data)

Installation

****Step 1: Clone the Repository****

Open your terminal and navigate to where you want to install SLATE:

```
cd /path/to/your/directory git clone  
https://github.com/Tilanthi/SLATE.git cd SLATE
```

****Step 2: Install Dependencies****

SLATE requires several Python packages. Install them using pip:

```
pip install -r requirements.txt
```

If you don't have a requirements.txt file, install the core dependencies manually:

```
pip install fastapi uvicorn numpy pandas sqlite3 pip install aiohttp  
asyncio logging
```

****Step 3: Verify Installation****

Check that SLATE is properly installed:

```
python3 -c "import slate_core; print('SLATE installed successfully!')"
```

Starting SLATE

****Option 1: Start with Default Settings****

The simplest way to start SLATE:

```
python3 -m slate_core.server
```

SLATE will:

- Start the web server on port 8788
- Automatically begin discovering and testing strategies
- Be accessible at <http://localhost:8788>

****Option 2: Start in Background****

To run SLATE in the background (continues running after you close the terminal):

```
python3 -m slate_core.server > slate.log 2>&1 &
```

****Option 3: Start with Custom Configuration****

You can customize SLATE's behavior by editing configuration parameters before starting.

First Launch Checklist

When you first start SLATE, here's what to check:

- ****Server Started Successfully****: You should see "SLATE Server started successfully" in the terminal
- ****Dashboard Accessible****: Open your browser and go to <http://localhost:8788> - you should see the main dashboard
- ****Discovery Systems Running****: Check that both "Self-Evolving Discovery" and "Realistic Discovery" show as running
- ****Tests Being Generated****: Within a few minutes, you should see the "Total Tests" metric increasing
- ****No Error Messages****: Check the terminal output for any error messages (warnings are usually okay)

Stopping SLATE

To stop SLATE gracefully:

****If running in foreground****: Press `Ctrl+C` in the terminal

****If running in background****:

```
pkill -f "slate_core.server"
```

4. The Dashboard Interface

Dashboard Overview

The SLATE dashboard is your main interface for monitoring and interacting with the system. Access it at <http://localhost:8788/dashboard> once SLATE is running.

Main Sections

****System Metrics Bar**** (Top)

Shows key performance indicators at a glance:

- ****Candidates****: Number of strategy candidates generated
- ****Deployed****: Strategies that passed validation
- ****Total Tests****: Realistic backtests completed
- ****Success Rate****: Percentage of candidates that passed validation
- ****Generation****: Current evolution generation number
- ****Best Sharpe****: Highest Sharpe ratio achieved

****Brain Visualization**** (Upper Left)

An interactive neural network display showing SLATE's current thinking:

- Nodes represent strategy types and market conditions
- Connections show relationships between components
- Colors indicate activity levels
- Updates in real-time as SLATE discovers new patterns

****Performance Charts**** (Upper Right)

Six mini-charts showing different aspects of system performance:

- ****Skills****: Current capability levels in different areas
- ****Returns****: Recent strategy returns over time
- ****Discovery****: Discovery rate and diversity
- ****Drawdown****: Maximum drawdown patterns
- ****Sharpe Ratio****: Performance efficiency over time
- ****Win Rate****: Percentage of profitable trades

****Discovery Status Panel**** (Middle Left)

Real-time information about discovery activities:

- Current cycle number
- Number of strategies being tested
- Discovery phase (generate, test, evaluate, evolve)
- Recent findings and insights

****Activity Stream**** (Middle Right)

A live feed of system activities:

- Strategies deployed
- Tests completed
- Performance milestones
- System events and notifications

****Decision Log**** (Bottom Left)

Detailed records of SLATE's decisions:

- Why certain strategies were chosen for testing
- Performance evaluation results
- Evolution decisions and rationale

****Control Panel**** (Bottom Right)

Quick actions to control SLATE:

- Pause/Resume discovery
- Adjust discovery speed
- Trigger specific test types
- Export results

Navigation Menu

****Home****: Return to main dashboard

****Discovery****: View detailed discovery statistics and results

****Strategies****: Browse all tested strategies with performance metrics

****Analytics****: Advanced analytics and performance reports

****Settings****: Configure system parameters and preferences

Understanding the Color Scheme

SLATE uses colors to convey important information:

****Cyan (Blue-Green)****: Normal operation, healthy system state

****Amber (Orange)****: Warning or caution, attention needed

****Emerald (Green)****: Success, profitable strategies

****Coral (Red)****: Poor performance, failed tests

****Violet (Purple)****: Learning activities, evolution processes

Dashboard Interactivity

****Click on Metrics****: Most metrics are clickable and will show detailed breakdowns

****Hover Over Charts****: Displays exact values and timestamps

****Activity Stream Items****: Click to see full details of any activity

****Brain Nodes****: Click to focus on specific strategy types or market conditions

****Refresh****: Dashboard auto-refreshes every 10 seconds, or manually refresh with F5

5. Discovery Systems

SLATE employs two complementary discovery systems that work together to find and test profitable trading strategies.

Self-Evolving Discovery System

****Purpose****: Generate new trading strategies by learning from past successes

****How It Works****:

The self-evolving system uses a genetic algorithm approach:

- ****Analyze Database****: Reviews all past test results to identify patterns in what works
- ****Select Parents****: Chooses the best-performing strategies as "parents" for the next generation
- ****Generate Offspring****: Creates new strategies by combining and mutating parameters from parent strategies

- ****Validate Candidates****: Applies quick validation filters to eliminate obviously flawed candidates

- ****Queue for Testing****: Sends promising candidates to the realistic discovery system for thorough testing

****Discovery Methods****:

Parameter Variation: Takes a successful strategy and slightly adjusts its parameters to see if small improvements are possible

Signal Combination: Combines multiple strategy types (like momentum + mean reversion) to create hybrid approaches

Regime Adaptation: Adapts strategies for specific market conditions (trending, ranging, volatile, quiet)

Multi-Timeframe: Creates strategies that analyze multiple timeframes simultaneously

Ensemble Learning: Combines multiple strategies into a voting system for more robust signals

****Skill Evolution****:

The system tracks its own skills in different areas:

- ****Pattern Recognition****: Ability to identify profitable patterns

- ****Anomaly Detection****: Finding unusual market conditions

- ****Adaptive Optimization****: Adjusting strategies based on performance

- ****Causal Inference****: Understanding cause-effect relationships

- ****Meta-Learning****: Learning how to learn better

These skills improve over time as the system discovers what works.

Realistic Discovery System

****Purpose****: Test strategies against real historical market data under realistic conditions

****How It Works****:

- ****Generate Strategies****: Creates strategies systematically across all strategy types

- ****Fetch Historical Data****: Retrieves real market data including price, volume, and order book information

- ****Run Backtests****: Simulates trading using the strategy on historical data

- **Calculate Metrics**: Computes performance metrics like returns, Sharpe ratio, maximum drawdown

- **Store Results**: Saves complete test results with all performance data

Testing Features:

Realistic Market Conditions:

- Accounts for trading fees
- Models slippage (price movement during order execution)
- Uses real historical price data
- Simulates order book depth

Multi-Path Testing:

- Tests strategies across multiple possible market scenarios
- Uses bootstrap resampling to create alternative price paths
- Calculates confidence intervals for performance metrics
- Identifies strategies robust across different scenarios

Systematic Coverage:

- Tests all 10 strategy types evenly
- Rotates through different timeframes
- Ensures comprehensive exploration of strategy space

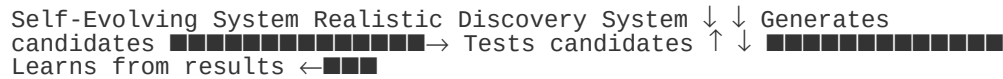
Performance Metrics Calculated:

- **Total Return**: Overall profit or loss percentage
- **Sharpe Ratio**: Risk-adjusted return (higher is better)
- **Sortino Ratio**: Return relative to downside risk
- **Maximum Drawdown**: Largest peak-to-trough decline
- **Win Rate**: Percentage of profitable trades
- **Profit Factor**: Ratio of profits to losses
- **Calmar Ratio**: Return relative to maximum drawdown

- **Equity Curve Smoothness**: Consistency of equity growth

The Feedback Loop

The true power of SLATE comes from how these two systems work together:



Flow:

- Realistic Discovery tests strategies and stores results in database
- Self-Evolving reads database to learn what worked
- Self-Evolving generates new candidates based on successful patterns
- New candidates queued for realistic testing
- Cycle repeats, continuously improving

Benefits:

- **Continuous Improvement**: Each cycle builds on knowledge from previous cycles
- **Diversified Exploration**: Explores different approaches systematically
- **Adaptive Focus**: Spends more time on strategy types that show promise
- **Memory Retention**: Never forgets what it has learned

Queue Management

The candidate queue manages the flow between systems:

- **Capacity**: Holds up to 100 candidates
- **Priority**: No priority system - first in, first out
- **Statistics**: Tracks how many candidates from each source
- **Monitoring**: Real-time queue size and processing status

You can check queue status in the dashboard or via the API.

6. Strategy Types Explained

SLATE tests ten different strategy types, each based on different market theories and approaches. Understanding these types will help you interpret results and focus on what matters most for your trading goals.

1. Momentum Strategies

Theory: Assets that have performed well recently will continue to perform well in the near future.

How It Works:

- Calculates rate of change (ROC) over a specified period
- Buys when ROC exceeds a threshold (positive momentum)
- Sells when ROC falls below a threshold (negative momentum)
- Uses different lookback periods to capture different timeframes

Parameters:

- **Period**: Number of bars to look back (typically 10-50)
- **Threshold**: Minimum ROC to trigger signals (typically 1-5%)

Best Markets: Trending markets with clear direction

Example: "If BTC has risen 3% in the last 20 minutes, buy and hold for 5 minutes"

2. Mean Reversion Strategies

Theory: Prices will tend to return to their mean value over time.

How It Works:

- Calculates moving average and standard deviation over a period
- Identifies when price deviates significantly from the mean
- Buys when price is below the lower Bollinger Band (oversold)
- Sells when price is above the upper Bollinger Band (overbought)

Parameters:

- **Period**: Number of bars for calculation (typically 10-30)
- **Standard Deviations**: How far from mean to trigger signals (typically 1.5-2.5)

Best Markets: Ranging markets with no clear trend

Example: "If ETH price falls 2 standard deviations below its 20-period mean, buy expecting reversion"

3. Breakout Strategies

Theory: When price breaks through significant levels, it will continue in that direction.

****How It Works****:

- Tracks highs and lows over a specified period
- Generates buy signal when price breaks above recent high
- Generates sell signal when price breaks below recent low
- Often uses volume confirmation

****Parameters****:

- ***Period***: Lookback period for high/low (typically 15-30)
- ***Confirmation***: Whether to require volume confirmation (true/false)

****Best Markets****: Markets with low volatility followed by explosive moves

****Example****: "If BTC breaks above its highest price in the last hour, buy expecting continuation"

4. Trend Following Strategies

****Theory****: Identify established trends and ride them until they reverse.

****How It Works****:

- Uses moving average crossovers to identify trend direction
- Fast moving average crossing above slow = uptrend
- Fast moving average crossing below slow = downtrend
- Holds position while trend remains intact

****Parameters****:

- ***Fast Period***: Short-term moving average (typically 10-20)
- ***Slow Period***: Long-term moving average (typically 30-50)

****Best Markets****: Markets with sustained, multi-hour trends

****Example****: "When 10-period MA crosses above 30-period MA, buy and hold while cross remains valid"

5. Statistical Arbitrage Strategies

****Theory****: Exploits statistical relationships between related assets or time periods.

****How It Works****:

- Identifies cointegrated relationships or mean-reverting patterns
- Takes positions when statistical relationships deviate from expected values
- Uses statistical tests to determine significance

Parameters:

- **Lookback Period:** Period for statistical calculations (typically 20-50)
- **Entry Threshold:** Z-score threshold for entry (typically 1.5-2.5)
- **Exit Threshold:** Z-score threshold for exit (typically 0-0.5)

Best Markets: Markets with stable statistical relationships

Example: "When BTC/ETH ratio deviates 2 standard deviations from mean, long the underperformer"

6. Machine Learning Strategies

Theory: Uses ML models to predict price movements based on historical patterns.

How It Works:

- Trains models on historical data to recognize patterns
- Uses features like price changes, volume, technical indicators
- Generates predictions for future price movements
- Trades based on prediction confidence

Parameters:

- **Lookback Period:** Amount of history to consider (typically 20-100)
- **Prediction Horizon:** How far ahead to predict (typically 1-10 periods)
- **Model Type:** Type of ML algorithm (varies)

Best Markets: Markets with complex, non-linear patterns

Example: "ML model predicts 85% probability of price increase in next 5 minutes based on pattern recognition"

7. Regime Switching Strategies

Theory: Markets go through different regimes (trending, ranging, volatile) and strategies should adapt.

****How It Works**:**

- Identifies current market regime using statistical tests
- Switches between different sub-strategies based on regime
- Uses Hidden Markov Models or similar techniques
- Optimizes parameters for each regime

****Parameters**:**

- ***Regime Detection Method***: How to identify regimes (HMM, rules-based, etc.)
- ***Adaptation Speed***: How quickly to switch regimes (0.1-1.0)
- ***Target Regime***: Which regime to specialize in (if focusing)

****Best Markets****: All markets, but particularly those with distinct regimes

****Example****: "Detect trending regime → use momentum rules. Detect ranging regime → use mean reversion rules"

8. Order Flow Strategies

****Theory****: Analyzes the flow of buy/sell orders to predict price movements.

****How It Works**:**

- Examines order book imbalances
- Tracks large trades (whales)
- Measures buy/sell pressure
- Trades in direction of net order flow

****Parameters**:**

- ***Depth***: How deep in order book to look (typically 5-20 levels)
- ***Threshold***: Minimum imbalance to trigger signal (typically 60-70%)
- ***Time Window***: Period to aggregate order flow (typically 1-5 minutes)

****Best Markets****: Markets with transparent order books and high liquidity

****Example****: "If buy volume exceeds sell volume by 70% in last 2 minutes, buy expecting upward pressure"

9. Microstructure Strategies

Theory: Exploits patterns in market microstructure like bid-ask spreads and price impact.

How It Works:

- Analyzes tick-by-tick data
- Measures market impact of trades
- Identifies liquidity patterns
- Exploits temporary mispricings

Parameters:

- **Tick Sensitivity**: How much weight to give individual ticks (0.1-1.0)
- **Spread Threshold**: Minimum spread to consider (varies by asset)
- **Impact Window**: Time to measure price impact (typically 10-100 ticks)

Best Markets: Highly liquid markets with tight spreads

Example: "Detect temporary spread widening in ETH/USDT, buy when spread narrows expecting price appreciation"

10. Multi-Timeframe Strategies

Theory: Analyzes multiple timeframes simultaneously for more robust signals.

How It Works:

- Monitors trend on higher timeframe
- Looks for entry signals on lower timeframe
- Only takes trades in direction of higher timeframe trend
- Uses exponential weighting for recent timeframes

Parameters:

- **Timeframes**: Which timeframes to monitor (e.g., 1m, 5m, 15m, 1h)
- **Base Timeframe**: Primary timeframe for signals (typically 5m)
- **Weighting Method**: How to weight different timeframes (equal, exponential)

Best Markets: All markets, particularly those with trends on multiple timeframes

Example: "1h chart shows uptrend → only take buy signals from 5m chart → ignore sell signals"

Strategy Selection Guide

For Trending Markets:

- Momentum - Best for strong trends
- Trend Following - Ideal for sustained trends
- Breakout - Good for trend start
- Multi-Timeframe - Confirms trends across timeframes

For Ranging Markets:

- Mean Reversion - Classic range trading
- Statistical Arbitrage - Exploits range-bound relationships
- Machine Learning - Adapts to range conditions

For Volatile Markets:

- Regime Switching - Adapts to volatility changes
- Order Flow - Tracks money flow in volatility
- Microstructure - Exploits volatility-induced inefficiencies

For Beginners:

- Start with Momentum or Mean Reversion (easiest to understand)
- Add Trend Following once comfortable with trends
- Experiment with Multi-Timeframe for more robust signals

For Advanced Users:

- Machine Learning for pattern recognition
- Statistical Arbitrage for sophisticated relationships
- Regime Switching for adaptive approaches

Interpreting Strategy Performance

When reviewing results, consider:

****Consistency vs. Home Runs****: A strategy with consistent small gains may be better than one with occasional huge wins but frequent losses.

****Market Conditions****: A strategy that performs poorly overall might excel in specific market conditions you expect to see more of in the future.

****Risk-Adjusted Returns****: High returns with high drawdown may not be worth it. Look at Sharpe ratio and maximum drawdown.

****Sample Size****: Strategies tested on fewer data points may be lucky rather than good. Prioritize strategies with more test data.

7. Testing Methodologies

SLATE employs multiple testing methodologies to ensure strategies are robust and reliable. Understanding these approaches will help you properly evaluate strategy potential.

Single-Path Testing

****What It Is****:

Testing a strategy against one specific sequence of historical price data.

****How It Works****:

- Select a historical time period
- Apply strategy rules to each bar in sequence
- Simulate trades based on signals
- Calculate performance metrics

****Pros****:

- Simple and intuitive
- Fast to execute
- Easy to understand
- Good for initial screening

****Cons****:

- Only one possible outcome
- Sensitive to specific historical path

- May not represent future performance
- Can be misleading if historical period was unusual

****When to Use**:**

- Initial strategy screening
- Quick comparison between strategies
- Understanding basic strategy behavior
- Educational purposes

****Interpreting Results**:**

- Take single-path results as preliminary
- Look for consistency across different time periods
- Be cautious of strategies that only work in specific conditions

Multi-Path Testing

****What It Is**:**

Testing a strategy across multiple alternative price paths created through bootstrapping.

****How It Works**:**

- Take historical price data
- Break it into blocks (e.g., 100 consecutive bars)
- Randomly rearrange blocks to create new price paths
- Test strategy on each alternative path
- Calculate statistics across all paths

****Pros**:**

- Tests robustness across scenarios
- Provides confidence intervals
- Identifies strategy sensitivity to path
- More realistic assessment of risk

****Cons**:**

- Computationally intensive
- More complex to interpret
- Creates artificial price sequences
- Requires more data

****When to Use**:**

- Validating promising strategies
- Before deploying real capital
- Understanding strategy risk
- Comparing similar strategies

****Interpreting Results**:**

- Look at mean performance across paths
- Pay attention to worst-case scenarios
- Check if confidence intervals are acceptable
- Prefer strategies with consistent performance across paths

****Example Interpretation**:**

Strategy A: Mean return 5%, Std dev 2%, Best 8%, Worst 2% Strategy B: Mean return 5%, Std dev 10%, Best 20%, Worst -15% Strategy A is preferable despite same mean return because it's more consistent.

Bootstrap Engine

****What It Is**:**

A sophisticated system for creating realistic alternative price paths using block bootstrapping.

****How It Works**:**

- Divides historical data into blocks (preserves short-term dependencies)
- Randomly samples blocks to create new paths
- Maintains statistical properties of original data
- Creates hundreds or thousands of alternative scenarios

****Key Features**:**

- **Block Size**: Typically 50-100 consecutive bars
- **Preservation**: Maintains volatility clustering, autocorrelation
- **Realism**: Creates paths that look like real market data
- **Diversity**: Generates wide range of possible scenarios

Block Sizes Explained:

- **Small blocks** (20-50): More variation, less realistic short-term
- **Medium blocks** (50-100): Balance between variation and realism
- **Large blocks** (100-200): Very realistic, less variation

When to Use:

- Final validation before live trading
- Stress testing strategies
- Calculating Value at Risk (VaR)
- Understanding tail risk

Performance Metrics Explained

SLATE calculates numerous metrics to evaluate strategy performance:

Return Metrics:

- **Total Return**: Overall profit/loss percentage
- **Mean Return**: Average return across multiple tests
- **Median Return**: Middle return (less sensitive to outliers)
- **Best/Worst Return**: Range of outcomes

Risk Metrics:

- **Maximum Drawdown**: Largest peak-to-trough decline
- **Volatility**: Standard deviation of returns
- **Value at Risk (VaR)**: Loss at 95% confidence
- **Conditional VaR**: Average loss in worst 5% of cases

Risk-Adjusted Return Metrics:

- ***Sharpe Ratio***: Return per unit of risk (higher is better)

- Above 2: Excellent

- 1-2: Good

- 0.5-1: Acceptable

- Below 0.5: Poor

- ***Sortino Ratio***: Return relative to downside risk only

- ***Calmar Ratio***: Return relative to maximum drawdown

****Consistency Metrics****:

- ***Win Rate***: Percentage of profitable trades

- ***Profit Factor***: Gross profit / Gross loss

- ***Equity Curve Smoothness***: How consistently equity grows

- ***Consistency Ratio***: How often strategy is profitable across tests

****Robustness Metrics****:

- ***Robustness Score***: Combined measure of return and consistency

- ***Confidence Intervals***: Range of likely values

- ***Success Rate in Multi-Path***: How often profitable across paths

Interpreting Confidence Intervals

Multi-path testing generates confidence intervals:

****95% Confidence Interval****: We're 95% confident the true value lies in this range

****Example****:

Sharpe Ratio: 1.5 [1.2, 1.8] Interpretation: Best estimate is 1.5, but true value likely between 1.2 and 1.8

****Narrow Intervals****: More confidence in estimate (better)

****Wide Intervals****: Less certainty (more testing needed)

****Intervals Including Zero****: Effectively no edge (avoid these strategies)

Realistic Assumptions

All SLATE testing incorporates realistic trading assumptions:

****Trading Costs**:**

- Maker fees: 0.02% (taker fees higher)
- Financing costs for leveraged positions
- Spread costs automatically considered

****Slippage**:**

- Orders don't execute at exact signal price
- Larger positions experience more slippage
- Market impact modeled realistically

****Execution**:**

- Orders execute when liquidity available
- Partial fills possible
- Order book depth considered

****Position Limits**:**

- Maximum position size enforced
- Risk limits respected
- Realistic capital constraints

Sample Size Considerations

****Minimum Tests**:**

- Single-path: At least 100 trades
- Multi-path: At least 100 paths

****Statistical Significance**:**

- More data = more confidence
- Be cautious of small samples
- Results can be due to chance with few tests

****Time Period**:**

- Longer periods = more diverse market conditions

- Minimum 30 days recommended
- Multiple market cycles ideal

8. Practical Usage Examples

This section provides natural language examples of how to use SLATE for different trading goals and scenarios.

Example 1: Finding a Simple Momentum Strategy

Goal: "I want to find a basic momentum strategy that works well in trending markets for Bitcoin"

How SLATE Helps:

- **Start the System:** Launch SLATE and let it run for several hours to build up test results
- **Access the Dashboard:** Go to <http://localhost:8788/dashboard>
- **Filter by Strategy Type:** Look for "momentum" in the Strategy Types section
- **Review Performance:**
 - Look at the Performance Metrics panel
 - Find momentum strategies with high Sharpe ratios
 - Check maximum drawdown to ensure risk is acceptable
- **Examine Specific Strategies:**
 - Click on a promising momentum strategy
 - View its parameters (lookback period, threshold)
 - Check performance across different timeframes
- **Analyze Results:**
 - Look for strategies tested on multiple days
 - Prefer those with consistent performance
 - Check if strategy performs well in both uptrending and downtrending markets

Expected Outcome:

After letting SLATE run for 24-48 hours, you might find something like:

Strategy: momentum_15m_45 Parameters: Period=20, Threshold=2.5%
Performance: Sharpe=1.2, Max Drawdown=-8%, Win Rate=55% Best in:
Trending markets, 15-minute timeframe

Natural Language Query: "Show me the best momentum strategies for Bitcoin that have been tested on at least 100 trades"

Example 2: Creating a Mean Reversion Strategy for Ethereum

Goal: "I want a strategy that buys when Ethereum drops too far below its average and sells when it rises above"

How SLATE Helps:

- **Let SLATE Discover:** The system automatically tests mean reversion strategies
- **Focus on ETH:**
 - Check the Strategy Types panel
 - Look for "mean_reversion" strategies
 - Filter by ETH/USDT pair if available
- **Examine Parameters:**
 - Look for strategies with periods around 20-30 (typical for mean reversion)
 - Check standard deviation thresholds (typically 1.5-2.5)
 - Find strategies that balance sensitivity with reliability
- **Check Robustness:**
 - Prefer strategies tested across multiple market conditions
 - Look at worst-case scenarios
 - Ensure strategy doesn't fail in volatile markets

Expected Outcome:

Strategy: mean_reversion_5m_22 Parameters: Period=25, StdDev=2.0
Performance: Sharpe=1.8, Max Drawdown=-5%, Win Rate=62% Best in:
Ranging markets, 5-minute timeframe Not suitable for: Strong trending
markets

Natural Language Query: "Find mean reversion strategies for Ethereum with low drawdown that work well in ranging markets"

Example 3: Combining Multiple Strategy Types

Goal: "I want to diversify by using multiple strategy types together"

How SLATE Helps:

- **Review Top Performers by Type**:
- Check each strategy type separately
- Identify the best 2-3 strategies from each type
- Look for uncorrelated performance
- **Check Correlation**:
- Use the Analytics section to see correlation between strategies
- Look for strategies that perform well in different conditions
- Ensure they don't all fail at the same time
- **Create a Portfolio**:
- Select 3-5 strategies from different types
- Allocate capital based on risk-adjusted returns
- Monitor combined performance

Example Portfolio:

30%: Momentum (for trending markets) 25%: Mean Reversion (for ranging markets) 20%: Trend Following (for sustained moves) 15%: Breakout (for volatile breakouts) 10%: Multi-Timeframe (for confirmation)

Natural Language Query: "Show me a diversified portfolio combining the best strategies from different types"

Example 4: Finding Strategies for Specific Market Conditions

Goal: "I want strategies that work well when the market is very volatile"

How SLATE Helps:

- **Identify Volatile Periods**:
- Look at the market data to find volatile times
- Check which strategies performed best then
- **Focus on Appropriate Types**:
- Regime Switching (adapts to volatility)
- Order Flow (tracks money in volatile conditions)
- Microstructure (exploits volatility inefficiencies)

• **Review Performance**:

- Check how strategies performed during known volatile periods
- Look at multi-path results for worst-case scenarios
- Ensure strategies handle extreme volatility

Expected Outcome:

Strategy: regime_switching_1m_8 Parameters: Adaptation Speed=0.7,
Regime Detection=HMM Performance in Volatility: Sharpe=2.1, Max
Drawdown=-12% Specializes in: High volatility regimes Also performs:
Acceptably in other conditions

Natural Language Query: "Which strategies perform best during high volatility periods with acceptable drawdown?"

Example 5: Evolving an Existing Strategy

Goal: "I have a strategy concept but want to improve it automatically"

How SLATE Helps:

• **Start with Concept**:

- Your base strategy: "Buy when 20-period ROC > 3%, sell when ROC < -2%"

• **Let SLATE Explore**:

- SLATE automatically tests variations of your concept
- Tests different lookback periods (15, 20, 25, 30)
- Tests different thresholds (2%, 2.5%, 3%, 3.5%)
- Tests different timeframes (1m, 5m, 15m)

• **Review Improvements**:

- Compare evolved versions to original
- Look for statistically significant improvements
- Check if improvements hold across different market conditions

Evolution Results:

Original: Sharpe=1.0, Max Drawdown=-10% Evolved v1: Sharpe=1.2, Max
Drawdown=-9% (Period=22, Threshold=2.8%) Evolved v2: Sharpe=1.5, Max
Drawdown=-7% (Period=18, Threshold=3.2%) Best: Evolved v2 with 50%
improvement in risk-adjusted returns

Natural Language Query: "Take my momentum strategy and find the best parameter combinations through evolution"

Example 6: Finding Strategies with Low Drawdown

Goal: "I need strategies that don't lose much money even in bad conditions"

How SLATE Helps:

- **Filter by Drawdown:**
- Use the dashboard to sort by Maximum Drawdown
- Look for strategies with drawdown under 10%
- Check if returns are still acceptable
- **Check Consistency:**
- Review worst-case scenarios in multi-path tests
- Ensure strategy doesn't have occasional large losses
- Look at equity curve smoothness
- **Validate with Realistic Testing:**
- Ensure drawdown calculated with realistic assumptions
- Check performance in different market regimes
- Verify results aren't due to lucky timing

Expected Outcome:

```
Strategy: trend_following_1h_12 Parameters: Fast=15, Slow=35
Performance: Sharpe=1.1, Max Drawdown=-6%, Win Rate=52%
Characteristics: Slow but steady, minimal large losses Best for:
Risk-averse traders, longer timeframes
```

Natural Language Query: "Find all strategies with maximum drawdown under 8% that still have positive returns"

Example 7: Quick Strategy Screening

Goal: "I want to quickly see what's working right now without spending hours analyzing"

How SLATE Helps:

- **Check Dashboard Overview:**
- Look at System Metrics for overall health
- Check Recent Discoveries for latest findings

- Review Activity Stream for what's happening
- ****Focus on Top Performers****:
- Use the "Top Strategies" panel
- Review best Sharpe ratios across all types
- Check if any strategy types stand out
- ****Quick Validation****:
- Ensure strategies have sufficient test data
- Check if results are statistically significant
- Look for consistency across time periods

****5-Minute Review Process****:

1. Open dashboard 2. Check "Best Sharpe" metric - note value 3. Review "Top Strategies" panel - note types 4. Check "Total Tests" - ensure sufficient data 5. Identify 2-3 promising strategies 6. Click each for details 7. Note parameters and performance 8. Done! You know what's working

****Natural Language Query****: "Give me a quick summary of the best performing strategies tested so far"

Example 8: Long-Term Strategy Development

****Goal****: "I want to spend a week developing and refining a comprehensive trading approach"

****How SLATE Helps****:

****Day 1-2: Discovery****

- Let SLATE run and build test database
- Review all strategy types
- Identify promising areas
- Note any surprises or unexpected findings

****Day 3-4: Analysis****

- Deep dive into top 5-10 strategies
- Analyze parameter sensitivities
- Check robustness across conditions

- Understand why strategies work

****Day 5-6: Refinement****

- Focus on most promising strategy types
- Let evolution run longer
- Test variations and combinations
- Validate with multi-path testing

****Day 7: Selection****

- Choose final strategies
- Create diversified portfolio
- Document approach and expectations
- Plan implementation

****Natural Language Query**:** "I want to spend a week developing a complete trading system using SLATE - guide me through the process"

Example 9: Backtesting a Strategy Idea

****Goal**:** "I have a strategy idea and want to see if it works historically"

****How SLATE Helps**:**

- ****Describe Your Strategy**:**
 - "Buy when RSI < 30 and price is below 200-period MA"
 - "Sell when RSI > 70 or price crosses above MA"
- ****Let SLATE Test Similar Concepts**:**
 - SLATE likely has tested similar mean reversion strategies
 - Check results for strategies using RSI or moving averages
 - Look at parameter ranges tested
- ****Compare Your Approach**:**
 - See if your parameters are in the optimal range
 - Check if similar strategies performed well

- Identify potential improvements

****Expected Results****:

Similar Strategies Found: - mean_reversion_1m_45: Uses RSI threshold, Sharpe=1.3 - mean_reversion_5m_12: Uses MA filter, Sharpe=1.5 - ensemble_3m_8: Combines RSI + MA, Sharpe=1.8 Your Strategy: RSI<30 + price<MA SLATE Findings: Good concept, consider: - Using 5-minute timeframe - Adding volume confirmation - Optimizing RSI period (14 vs 20)

****Natural Language Query****: "Test my strategy idea of buying when RSI is below 30 and price is below the 200-period moving average"

Example 10: Understanding Why a Strategy Failed

****Goal****: "I tried a strategy that looked great in testing but failed in live trading - why?"

****How SLATE Helps****:

- ****Review Test Conditions****:
- Check if SLATE tested similar strategies
- Look at their performance metrics
- Identify any differences in approach
- ****Analyze Multi-Path Results****:
- Check confidence intervals
- Look at worst-case scenarios
- See if live result falls within expected range
- ****Identify Potential Issues****:
- ****Overfitting****: Strategy too specific to historical data
- ****Look-Ahead Bias****: Using future data in past
- ****Unrealistic Assumptions****: Ignoring costs, slippage
- ****Sample Size****: Too few tests for significance
- ****Regime Change****: Market conditions changed

****Common Problems****:

Issue: Strategy had 90% win rate in testing SLATE Finding: Only 20 trades tested - too small Solution: Need at least 100 trades for significance Issue: Strategy made money every month for 6 months SLATE Finding: All profit came from 2 lucky trades Solution: Remove outliers and re-evaluate Issue: Strategy worked great in 2023, failed in 2024 SLATE Finding: 2023 had unusual volatility, 2024 different Solution:

Focus on strategies robust across conditions

Natural Language Query: "Why did my backtested strategy fail when I traded it live?"

9. Understanding Results

Reading Performance Metrics

When reviewing SLATE test results, you'll encounter numerous metrics. Here's how to interpret them:

Sharpe Ratio (Most Important)

- Measures risk-adjusted return
- Above 2.0: Excellent, worth serious consideration
- 1.0-2.0: Good, acceptable for live trading
- 0.5-1.0: Marginal, proceed with caution
- Below 0.5: Poor, generally not worth using
- Negative: Strategy loses money on risk-adjusted basis

Maximum Drawdown

- Largest peak-to-trough equity decline
- Below 10%: Excellent risk control
- 10-20%: Acceptable for most traders
- 20-30%: High risk, only for experienced traders
- Above 30%: Extremely risky, generally avoid

Win Rate

- Percentage of profitable trades
- Above 60%: Excellent
- 50-60%: Good
- 40-50%: Acceptable if wins are larger than losses
- Below 40%: Concerning unless profit factor is very high

Profit Factor

- Ratio of gross profits to gross losses
- Above 2.0: Excellent
- 1.5-2.0: Good
- 1.0-1.5: Marginal
- Below 1.0: Strategy loses money

****Total Return****

- Overall profit/loss percentage
- Context matters: High return with high drawdown isn't good
- Compare to Sharpe ratio for risk-adjusted view

****Calmar Ratio****

- Return relative to maximum drawdown
- Above 3.0: Excellent
- 1.5-3.0: Good
- 0.5-1.5: Marginal
- Below 0.5: Poor

Interpreting Multi-Path Results

When SLATE runs multi-path tests, you'll see additional metrics:

****Mean vs. Median Return****

- Mean: Average return (can be skewed by outliers)
- Median: Middle return (more robust)
- Large difference suggests outliers are important

****Standard Deviation****

- Measures variability of returns
- Lower is better (more predictable)
- Use to assess consistency

****Confidence Intervals****

- 95% CI: Range containing true value 95% of the time
- Narrow intervals: More confidence
- Intervals containing zero: No statistically significant edge

****Worst-Case Scenario****

- Minimum return across all paths
- Important for risk assessment
- Can you tolerate this outcome?

****Consistency Ratio****

- Percentage of paths that were profitable
- Above 80%: Excellent
- 60-80%: Good
- 40-60%: Marginal
- Below 40%: Too unpredictable

Identifying Reliable Strategies

****Green Flags** (Good Signs):**

- High Sharpe ratio (>1.5) with reasonable drawdown ($<15\%$)
- Consistent performance across different time periods
- Good performance in multi-path testing (high consistency ratio)
- Narrow confidence intervals
- Tested on large sample (100+ trades or paths)
- Performs acceptably in different market regimes
- Logical explanation for why it works

****Red Flags** (Warning Signs):**

- Amazing backtest but no logical basis
- Only works in specific market conditions
- Very few trades or tests

- Huge profits with huge drawdowns
- Performance depends on 1-2 lucky trades
- Wide confidence intervals including zero
- Deteriorating performance in recent tests
- Overfitted to specific historical period

Comparing Strategies

When choosing between strategies:

Risk-Adjusted Returns: Prefer higher Sharpe ratio over higher total return

Consistency: Choose strategies with more predictable performance

Robustness: Favor strategies that work across different conditions

Simplicity: Simpler strategies often work better out-of-sample

Sample Size: More data trumps better-looking results with less data

Correlation: Choose uncorrelated strategies for diversification

Common Pitfalls

Overfitting: Strategy too specific to historical data

- Solution: Focus on strategies tested on diverse conditions

Look-Ahead Bias: Using future data unknowingly

- Solution: Ensure SLATE's realistic testing assumptions

Survivorship Bias: Only testing winners

- Solution: SLATE tests all generated strategies, avoiding this

Ignoring Costs: Not accounting for fees and slippage

- Solution: SLATE includes realistic trading costs

Small Sample Size: Drawing conclusions from too little data

- Solution: Wait for more tests before judging strategies

Regime Changes: Market conditions evolving

- Solution: Focus on adaptive, robust strategies

Making Trading Decisions

****Before Live Trading**:**

- Ensure strategy has sufficient test data (100+ trades)
- Check multi-path results show consistency
- Verify drawdown is acceptable
- Understand market conditions strategy prefers
- Have clear exit criteria

****Position Sizing**:**

- Base on maximum drawdown from testing
- Assume real results will be worse than backtest
- Start small and scale gradually
- Never risk more than 1-2% per trade

****Ongoing Monitoring**:**

- Track live performance vs. expected
- Watch for degradation in performance
- Be prepared to stop if results diverge from expectations
- Market conditions may change, affecting strategy performance

****When to Stop**:**

- Live performance much worse than expected
- Market conditions no longer suitable
- Better strategy becomes available
- Risk tolerance changes

10. Advanced Configuration

Customizing Discovery Parameters

SLATE can be customized for different use cases and preferences.

****Discovery Speed**:**

- **Fast**: Fewer tests, quicker results (good for exploration)
- **Balanced**: Moderate testing (recommended)
- **Thorough**: Extensive testing (slower but more reliable)

Strategy Focus:

- **Diversified**: Test all strategy types equally (default)
- **Targeted**: Focus on specific types (e.g., only momentum)
- **Adaptive**: Let SLATE focus on what works

Risk Tolerance:

- **Conservative**: Prioritize low drawdown strategies
- **Moderate**: Balance risk and return
- **Aggressive**: Maximize returns regardless of risk

Adjusting Validation Criteria

The self-evolving system uses validation criteria to filter candidates. You can adjust these:

Quick Validation:

```
# In self_evolution.py, modify _quick_validation method: if
candidate.expected_return < 0: # Minimum expected return return False
if candidate.max_drawdown < -50: # Maximum acceptable drawdown return
False if candidate.sharpe_ratio < 0.5: # Minimum Sharpe ratio return
False
```

Relax for More Exploration:

- Lower expected_return threshold
- Increase acceptable max_drawdown
- Reduce minimum sharpe_ratio

Tighten for Quality:

- Raise expected_return threshold
- Decrease acceptable max_drawdown
- Increase minimum sharpe_ratio

Configuring Realistic Testing

Historical Data Period:

- Default: 30 days of recent data
- Longer: More diverse conditions, slower testing
- Shorter: Faster testing, less diversity

****Testing Frequency**:**

- Default: Continuous cycles
- Can pause/resume as needed
- Adjust based on computational resources

****Multi-Path Testing**:**

- Default: Enabled for 20% of strategies
- Increase for more robust validation
- Disable for faster (but less reliable) testing

Database Management

****Automatic Archiving**:**

- Default: Every 50 cycles
- Keeps full detail for: Recent 100 + Best 100 (by Sharpe)
- Archives older results (summary stats only)

****Manual Cleanup**:**

```
# Access database cleanup endpoint curl -X POST http://localhost:8788/
api/realistic-discovery/database/cleanup?max_records=5000
```

****Database Size**:**

- Typical: 5-10MB with archiving enabled
- Without archiving: Can grow to hundreds of MB
- Monitor and clean as needed

Performance Tuning

****For Faster Discovery**:**

- Reduce number of workers
- Disable multi-path testing

- Shorten historical data period
- Tighter validation (fewer candidates to test)

****For Better Quality**:**

- Increase workers (if CPU available)
- Enable more multi-path testing
- Longer historical data period
- Looser validation (more candidates explored)

****Resource Requirements**:**

- Minimum: 2 CPU cores, 4GB RAM
- Recommended: 4+ CPU cores, 8GB+ RAM
- For extensive multi-path: 8+ CPU cores, 16GB+ RAM

11. Troubleshooting

Common Issues and Solutions

****SLATE Won't Start****

Symptoms: Server fails to start, error messages

- Check Python version (must be 3.8+)
- Verify all dependencies installed
- Check port 8788 not already in use
- Review error messages in terminal

Solution:

```
# Check Python version python3 --version # Install dependencies pip
install -r requirements.txt # Check for port conflicts lsof -i :8788 #
Kill existing process if needed pkill -f slate_core.server
```

****Dashboard Shows "No Results"****

Symptoms: Metrics all zero, no activity

- SLATE may have just started (wait 5-10 minutes)
- Discovery systems may not be running

- Database may be corrupted

Solution:

```
# Check if systems are running curl
http://localhost:8788/api/discovery/status # Restart SLATE pkill -f
slate_core.server python3 -m slate_core.server # Clear and restart if
corrupted rm slate_realistic_discoveries.db python3 -m
slate_core.server
```

****Tests Not Increasing****

Symptoms: Total test count stays same

- Discovery may have completed planned cycles
- Validation too strict (all candidates rejected)
- System paused or stopped

Solution:

```
# Check system status curl
http://localhost:8788/api/realistic-discovery/status curl
http://localhost:8788/api/discovery/status # Restart discovery curl -X
POST http://localhost:8788/api/realistic-discovery/start?cycles=10000
# Check validation settings in self_evolving.py
```

****Poor Performance Overall****

Symptoms: All strategies showing negative returns

- Market conditions may be challenging
- Parameters may be too restrictive
- Need longer testing period

Solution:

- Let SLATE run longer (days rather than hours)
- Review if specific strategy types perform better
- Consider if current market conditions favor any types
- Remember: not all periods are profitable for all strategies

****Database Too Large****

Symptoms: Database file growing to hundreds of MB

- Archiving may not be enabled
- Too many old results stored

- Need manual cleanup

Solution:

```
# Trigger manual archive curl -X POST
http://localhost:8788/api/realistic-discovery/database/cleanup # Or
delete and start fresh rm slate_realistic_discoveries.db python3 -m
slate_core.server
```

****Memory Issues****

Symptoms: System slow, crashes

- Too much data loaded
- Memory leak
- Insufficient RAM

Solution:

- Reduce historical data period
- Enable more aggressive archiving
- Close other applications
- Increase system RAM if possible

Getting Help

****Check Logs**:**

```
# Server logs tail -f slate_server.log # Discovery logs tail -f
slate_discovery.log
```

****API Status Checks**:**

```
# Overall health curl http://localhost:8788/health # Discovery status
curl http://localhost:8788/api/discovery/status | python3 -m json.tool
# Realistic discovery status curl
http://localhost:8788/api/realistic-discovery/status | python3 -m
json.tool # Database info curl
http://localhost:8788/api/realistic-discovery/database/info | python3
-m json.tool
```

****Community Resources**:**

- GitHub Issues: <https://github.com/Tilanthi/SLATE/issues>
- Documentation: Check project README and markdown files
- Example Configurations: Look in slate_core/config.py

12. Best Practices

For New Users

****Start Simple**:**

- Let SLATE run for at least 24 hours before making decisions
- Focus on understanding one or two strategy types first
- Don't override default settings until you understand them
- Start with paper trading, not real money

****Learn Gradually**:**

- Check dashboard daily to see patterns emerge
- Read about strategies that perform well
- Understand why certain strategies work better
- Experiment with filtering by strategy type

****Build Intuition**:**

- Notice which strategy types perform well in current market
- Watch how performance changes over time
- Observe how different market conditions affect strategies
- Develop your own preferences and risk tolerance

For Experienced Users

****Advanced Strategies**:**

- Combine insights from multiple strategy types
- Create custom ensembles based on correlation analysis
- Use multi-path results for robust validation
- Focus on strategies robust across market regimes

****Optimization Techniques**:**

- Let evolution run longer before selecting strategies
- Use tiered storage to manage large databases
- Adjust validation criteria based on market conditions
- Monitor feedback loop effectiveness

****Risk Management**:**

- Always diversify across strategy types
- Use maximum drawdown from testing to size positions
- Assume live results will be worse than backtest
- Have clear stop-loss criteria

General Best Practices

****Continuous Learning**:**

- Review new discoveries regularly
- Stay updated on market conditions
- Understand limitations of backtesting
- Learn from both successes and failures

****Data Management**:**

- Monitor database size and archive regularly
- Keep backup of important discoveries
- Document strategies you deploy
- Track live performance vs expectations

****System Maintenance**:**

- Restart SLATE weekly to clear memory
- Check logs for errors or warnings
- Update software when new versions available
- Monitor resource usage (CPU, RAM, disk)

****Emotional Discipline**:**

- Don't chase recent performance
- Stick to your risk tolerance
- Avoid overconfidence in backtest results
- Be prepared for losing periods

****Realistic Expectations****:

- No strategy works all the time
- Even good strategies have drawdowns
- Past performance doesn't guarantee future results
- Market conditions change over time

Conclusion

SLATE represents a powerful approach to automated trading strategy discovery and testing. By understanding its components and following best practices, you can leverage it to find strategies that match your trading goals and risk tolerance.

Remember:

- ****Let SLATE run long enough**** to generate meaningful results
- ****Focus on risk-adjusted returns**** rather than total returns
- ****Diversify across strategy types**** to reduce risk
- ****Validate thoroughly**** before risking real capital
- ****Monitor continuously**** and adapt as conditions change

The journey to profitable trading is a marathon, not a sprint. SLATE provides the tools to explore systematically and learn continuously. Your job is to interpret results wisely and trade with discipline.

Happy discovering!

Appendix A: Quick Reference

****Dashboard URL****: <http://localhost:8788/dashboard>

****API Endpoints****:

- Health: <http://localhost:8788/health>
- Discovery Status: <http://localhost:8788/api/discovery/status>
- Realistic Status: <http://localhost:8788/api/realistic-discovery/status>
- Queue Status: <http://localhost:8788/api/realistic-discovery/queue/status>

****Key Commands****:

```
# Start SLATE python3 -m slate_core.server # Stop SLATE pkill -f
slate_core.server # Check status curl http://localhost:8788/health #
View logs tail -f slate_server.log
```

****Performance Benchmarks**:**

- Sharpe > 2.0: Excellent
- Sharpe 1.0-2.0: Good
- Sharpe 0.5-1.0: Acceptable
- Max Drawdown < 10%: Excellent
- Max Drawdown 10-20%: Acceptable
- Win Rate > 60%: Excellent
- Win Rate 50-60%: Good

Appendix B: Glossary

****Alpha****: Return in excess of market benchmark

****Backtest****: Testing a strategy on historical data

****Bootstrap****: Resampling technique for creating alternative price paths

****Drawdown****: Peak-to-trough decline in equity

****Equity Curve****: Plot of account value over time

****Kelly Criterion****: Optimal position sizing formula

****Look-Ahead Bias****: Unintentional use of future data

****Multi-Path****: Testing across multiple price scenarios

****Overfitting****: Strategy too specific to historical data

****Regime****: Market condition (trending, ranging, volatile, etc.)

****Sharpe Ratio****: Risk-adjusted return measure

****Slippage****: Difference between expected and actual execution price

****Survivorship Bias****: Only considering winners, not losers

****Walk-Forward****: Rolling backtest validation technique

****Document Version****: 2.0

****Last Updated**:** April 15, 2026

****For SLATE Version**:** 2.0+

For the latest updates and community support, visit: <https://github.com/Tilanthi/SLATE>